

CSCE 627 THEORY OF COMPUTABILITY · MODULE 13 · LECTURE NOTES

The Limits, Honestly

What this course does and does not tell you.

MODULE OUTCOMES

- State what the results do and do not constrain.
- Avoid the standard overreaches.
- Explain how to use an impossibility result productively.
- Place this course relative to CSCE 637 and CSCE 658.
- State what a claim about computability can honestly assert.

1 What the semester establishes

- **There is a definition of algorithm**, it is robust across every reasonable model, and the coincidence of independent formalisations makes it almost certainly the right one (Modules 01, 04, 11).
- **A strict hierarchy of language classes** — regular, context-free, context-sensitive, recognisable, decidable — each separation proved, **with practical consequences at every level**: lexers, parsers, semantic analysis (Modules 02, 03).
- **Specific, natural, and practically important problems are undecidable** — halting, equivalence, reachability, aliasing — **and almost every semantic property of programs is, by Rice's theorem** (Modules 05, 06).
- **'Undecidable' names infinitely many distinct levels**, measured by quantifier alternation, **with the safety/liveness distinction as its most useful practical instance** (Module 07).
- **And no sound, complete, mechanically checkable formal system of sufficient strength exists** (Module 10) — **which is Module 05 again, in a different vocabulary. All of it permanent, and all of it independent of hardware, language, and era** — which no other course in this program can say about its contents.

2 The overreaches

The claim	What is wrong with it
'Static analysis is pointless, because the problems are undecidable.'	Undecidability constrains what a tool can <i>promise</i> , not whether it is <i>useful</i> (Modules 08, 12). Type checkers are undecidable-problem solvers and are used by everyone.

The claim	What is wrong with it
'Gödel shows human minds exceed machines.'	The argument assumes human reasoning is consistent and formalisable — which is precisely what would need to be established, so the argument assumes its conclusion (Module 10 §3).
'Quantum computers will break undecidability.'	A quantum computer computes exactly the same class of functions; only the speed of some computations changes (Module 11 §3).
'The halting problem means software cannot be verified.'	Software is verified routinely , by Module 12's four strategies — seL4, CompCert, and Airbus flight control are existence proofs.
'Undecidable means practically impossible.'	It conflates the universal quantifier with the typical case (Module 01 §3). Most instances of most undecidable problems are easy.
'AI cannot be genuinely intelligent, because of Gödel.'	The argument applies equally to any formalisable reasoner, including a human one — so either it proves too much or it assumes humans are not formalisable, which is the claim at issue.

Every overreach has the same shape: a result about a universally quantified claim is read as a result about practice, or about minds. Recognising the shape is quicker than refuting each instance, and you will meet all six.

THE MOST USEFUL CORRECTION

'No algorithm decides X for all inputs' and 'X is hard for the inputs I actually have' are different claims.

The first is proved. The second is an empirical question about your particular inputs, and it is **frequently false** — most real programs are obviously terminating, most real pointers obviously do not alias, and most real grammars are obviously unambiguous.

So an undecidability result tells you what a *total and exact* tool cannot promise, and nothing whatsoever about what a tool can deliver on your workload.

Which is why Module 12's strategies work at all, and why the correct response to an impossibility proof is to design the approximation rather than to abandon the problem. If one sentence from this course should survive, it is this one.

3 Using an impossibility result

- **Stop looking for the exact algorithm.** An hour spent proving impossibility saves a month of searching (Module 06 §4), and Rice's theorem frequently makes it ten minutes.
- **Identify what you actually need.** Frequently a sound approximation, or a correct answer on the inputs you have, is the entire requirement — and nobody asked for totality.
- **Choose from Module 12's four strategies,** and **state which of sound, complete, and terminating you gave up,** where users will see it.
- **Consider restricting the problem.** The most underused move, and the only one that removes the difficulty rather than managing it (Module 12 §2) — and it is available far more often than people assume.
- **And document the limit,** so the next person does not repeat your month. **The documentation step is the one that compounds: an impossibility recorded in a design document is worth considerably more than one proved and forgotten,** and it is the cheapest step on the list.

4 The semester, and the honest claim

Course	Asks	Answers take the form
CSCE 627 (this one)	Can it be done at all?	No algorithm exists. Proved, unconditional, permanent.
CSCE 637	Can it be done efficiently?	Not in polynomial time, unless $P = NP$ — mostly conditional on an open question.
CSCE 658	Does randomness help?	Sometimes, provably — and possibly never essentially, since whether randomness can always be removed is itself open.

Note the certainty gradient across the semester: this course's results are unconditional, CSCE 637's central claims are conditional on $P \neq NP$, and CSCE 658 contains an open question about whether its own subject matter is necessary (derandomisation). Which is the honest framing of a theory semester — the further you move from computability toward resources, the more the results depend on conjectures, and saying so is more useful than presenting all three as equally settled.

WHAT A CLAIM ABOUT COMPUTABILITY CAN HONESTLY ASSERT

'This problem is undecidable, by reduction from the halting problem; here is the reduction.'
Checkable by anyone, and the strongest form of claim available in this program.

'This property is semantic and non-trivial, so Rice's theorem applies.' Also fully checkable, and much faster to establish — and it is the claim you will make most often.

'Our analyser is sound and incomplete; here is a false positive we constructed, and here is the documented unsoundness source we accept.' The position stated and demonstrated rather than asserted.

And what you cannot honestly say: 'this is impossible', unqualified. **Impossible for a total, exact algorithm** — and that is a different and much narrower claim than the unqualified one, which is where every overreach in §2 begins.

WHERE THIS COURSE LEAVES YOU

You can prove a language non-regular, place a problem in the Chomsky hierarchy, construct a mapping reduction in the right direction, and apply Rice's theorem in under a minute — which is the technical content, and the last of those is the one you will use most.

You can derive Gödel's first incompleteness theorem from the halting problem, and state both theorems with their actual conditions rather than their popular versions — which is unusual and worth having.

And you can look at a tool and say which of sound, complete, and terminating it gave up. That is the practical skill of the course, and it changes how you read every analyser's output, every type error, and every 'no issues found'.

The closing rule is the program's, unchanged across twenty-two courses: state what you measured, state what you assumed, and never claim more than you established. In computability it means saying what the quantifier ranges over — because **the entire subject lives in the difference between 'for all inputs' and 'for my inputs',** and so does every mistake people make with it.

Where to see this taught differently

Original material only goes so far — hearing the same idea in another voice is how it sticks. These are free, and they cover this module specifically.

Resource	What it covers for this module
Franzén — Gödel's Theorem: An Incomplete Guide to Its Use and Abuse	§2's second and sixth rows, at book length — the standard correction, and worth reading before citing the theorems.
Aaronson — Why Philosophers Should Care About Computational Complexity (free)	The honest version of what these results do and do not bear on, spanning this course and CSCE 637.

Resource	What it covers for this module
Sipser — the whole book, reread	It is short, and the second reading is substantially different from the first — which is unusual for a textbook and is worth the weekend.
Rice's theorem, Module 06, reread	If one result from this course should be at your fingertips, it is this one — it answers 'is this analysis possible?' in ten minutes, repeatedly, for the rest of your career.

Exercises

Do these in code, not on paper, unless the exercise says otherwise. The module is not finished until they run.

- 1 **Write out the five established results** of §1 in your own words, with the module each came from.
- 2 **Find three of §2's overreaches in the wild** and write the correction for each.
- 3 **Explain the for-all-inputs versus for-my-inputs distinction** to someone who has not taken this course.
- 4 **Take an unsolved problem from your own work** and spend an hour on §3's first step.
- 5 **Whatever the outcome, document it** in a form the next person could use.
- 6 **For a tool you use daily, write its soundness position** in one sentence.
- 7 **Compare the three Semester 8 courses' claim forms** and say which you find most and least satisfying.
- 8 **Write the honest claim** for the undecidability result you proved in Project 1.
- 9 **Revisit what you wrote in Module 01's last exercise** and report what changed.
- 10 **Project 2 is now due.** Submit the tool, the undecidable problem identified with its argument, the soundness position, the false positive, the false negative, and your assessment of the documentation.

Self-check

Answer these in writing, from memory, without looking back at the notes. If you cannot, you have read this module but not yet learned it — that is useful information, not a failure.

- 1 Name the five results the semester established.
- 2 What makes them different from every other result in this program?
- 3 Give six overreaches and the correction to each.
- 4 What shape do all of them share?
- 5 State the for-all versus for-mine distinction and why it matters.
- 6 Give the five productive responses to an impossibility result.
- 7 Which compounds, and why?
- 8 Compare the three theory courses' claim forms.

- 9 Give three honest claims and the one thing you cannot say.
- 10 In this course, what does 'state what you assumed' mean?